WEEK – 5

Code:

```c
// CH.SC.U4CSE24165
#include <stdio.h>
#include <stdlib.h>

// Node structure
typedef struct Node {
    int key;
    struct Node *left, *right;
    int height;
} Node;

// Get height
int height(Node *n) {
    if (n == NULL) return 0;
    return n->height;
}

// Max function
int max(int a, int b) {
    return (a > b) ? a : b;
}

// Create node
Node* newNode(int key) {
    Node* node = (Node*)malloc(sizeof(Node));
    node->key = key;
    node->left = node->right = NULL;
    node->height = 1;
    return node;
}
```

```
// Right rotate
Node* rightRotate(Node* y) {
    Node* x = y->left;
    Node* T2 = x->right;

    x->right = y;
    y->left = T2;

    y->height = max(height(y->left), height(y->right)) + 1;
    x->height = max(height(x->left), height(x->right)) + 1;

    return x;
}

// Left rotate
Node* leftRotate(Node* x) {
    Node* y = x->right;
    Node* T2 = y->left;

    y->left = x;
    x->right = T2;

    x->height = max(height(x->left), height(x->right)) + 1;
    y->height = max(height(y->left), height(y->right)) + 1;

    return y;
}
```

```
// Get balance
int getBalance(Node* n) {
    if (n == NULL) return 0;
    return height(n->left) - height(n->right);
}

// Insert
Node* insert(Node* node, int key) {
    if (node == NULL) return newNode(key);

    if (key < node->key)
        node->left = insert(node->left, key);
    else if (key > node->key)
        node->right = insert(node->right, key);
    else
        return node;

    node->height = 1 + max(height(node->left), height(node->right)

    int balance = getBalance(node);

    // LL
    if (balance > 1 && key < node->left->key)
        return rightRotate(node);

    // RR
    if (balance < -1 && key > node->right->key)
        return leftRotate(node);

    // LR
    if (balance > 1 && key > node->left->key) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }
```

```c

    // RL
    if (balance < -1 && key < node->right->key) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }

    return node;
}

// Inorder
void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->key);
        inorder(root->right);
    }
}

// Main
int main() {
    int arr[] = {157,110,147,122,111,149,151,141,123,112,117,133};
    int n = 12;

    Node* root = NULL;

    for (int i = 0; i < n; i++)
        root = insert(root, arr[i]);

    printf("AVL Tree (Inorder): ");
    inorder(root);

    return 0;
}
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\Semester 4\Haskell\Capstone\Mini Project\Weather-App> gcc -mconsole 7.c -o 7 ; if ($?) { .\7.exe }
AVL Tree (Inorder): 110 111 112 117 122 123 133 141 147 149 151 157
PS D:\Semester 4\Haskell\Capstone\Mini Project\Weather-App>
```